

W12D03 — Health Check & Graceful Shutdown

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 27 May 2026

1. Mengapa Health Check Wajib Ada di Production

Tanpa health check endpoint, orchestrator (Docker, Railway, Kubernetes) tidak bisa membedakan apakah container sedang startup, sudah mati, atau berjalan tapi dependency-nya putus. Semua kondisi itu kelihatan sama dari luar — container berjalan, port terbuka — padahal isinya bisa sudah tidak berfungsi.

Analogi: seperti sistem absensi yang menelepon karyawan setiap 30 detik. Kalau tidak ada jawaban, supervisor dikirim. Kalau karyawan angkat tapi bilang 'saya sakit dan tidak bisa kerja', supervisor tidak kirim pelanggan ke meja itu.

2. /health vs /ready — Dua Pertanyaan Berbeda

Kesalahan paling umum adalah menyamakan kedua endpoint ini. Mereka punya tujuan, konsekuensi, dan isi yang berbeda.

Aspek	/health (liveness)	/ready (readiness)
Pertanyaan	Apakah proses masih hidup?	Apakah app siap terima traffic?
Kapan 200	Selama app bisa menjawab	Setelah semua dependency siap
Kapan 503	Hampir tidak pernah	Saat startup / dependency down
Konsekuensi fail	Orchestrator restart container	Load balancer skip instance ini
Cek dependency?	TIDAK — hanya konfirmasi hidup	YA — model, DB, disk, dll
Analogi	Apakah karyawan masih di kantor?	Apakah mejanya sudah siap?

Kenapa /health tidak boleh cek Redis? Karena konsekuensi gagalnya adalah container di-restart. Kalau Redis down, restart container tidak akan membantu — Redis tetap down. Yang perlu dilakukan adalah stop traffic ke instance ini (via /ready 503), bukan restart.

3. Graceful Shutdown — Berhenti dengan Rapi

Ketika container di-stop, Docker mengirim sinyal SIGTERM — ini adalah permintaan sopan: 'tolong bersiap untuk berhenti'. App punya waktu (default 10 detik) untuk menyelesaikan request yang sedang berjalan dan menutup resource. Setelah itu Docker kirim SIGKILL — paksa mati.

Analogi: kasir yang sedang melayani pelanggan. Graceful shutdown = bos bilang 'selesaikan pelanggan ini dulu, lalu tutup kasir'. Hard shutdown = langsung padamkan listrik di tengah transaksi.

Urutan shutdown yang benar:

1. **app_state.ready = False** — komunikasikan ke load balancer: stop kirim request baru
2. Tunggu request yang sedang berjalan selesai (FastAPI handle ini otomatis)
3. **Close database / Redis connection** — cegah connection leak
4. **Unload model** — bebaskan memori (penting saat restart cepat)
5. Log `app_shutdown_complete` — konfirmasi semua bersih

Kenapa ready=False HARUS jadi langkah pertama? Kalau diletakkan di akhir, ada jeda waktu di mana app sedang menutup resource tapi load balancer masih mengirim request baru. User akan menerima error karena

DB connection sudah ditutup.

4. lifespan — Kontrak Dua Arah di FastAPI

Sejak FastAPI 0.93+, cara lama `@app.on_event('startup')` sudah deprecated. Gantinya adalah lifespan context manager — startup dan shutdown ada di satu tempat, dipisahkan oleh `yield`.

Struktur lifespan:

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@appcontextmanager
async def lifespan(app: FastAPI):
    # ■■ STARTUP ■■
    app_state.model_service = load_model()
    app_state.redis_client = create_redis_client(...)
    app_state.ready = True          # selalu terakhir
    logger.info('app_ready')

    yield                          # app berjalan di sini

    # ■■ SHUTDOWN (dijamin oleh try/finally) ■■
    app_state.ready = False         # selalu pertama
    app_state.redis_client.close()
    app_state.model_service = None
    logger.info('app_shutdown_complete')

app = FastAPI(lifespan=lifespan)
```

	@app.on_event (deprecated)	lifespan (recommended)
Startup & shutdown	Dua fungsi terpisah	Satu context manager
Error handling	Tidak terjamin	try/finally — dijamin
Testing	Sulit di-mock	Mudah di-inject
Kejelasan kode	Tersebar di dua tempat	Satu tempat, urutan jelas

5. AppState — Singleton Pattern

AppState adalah dataclass yang menyimpan semua state global aplikasi. Diinstansiasi sekali di module level (singleton), lalu di-import ke mana saja yang butuh. Lebih type-safe dari dict karena akses via atribut (`app_state.ready`) bukan string key.

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class AppState:
    model_service: Optional[ModelService] = None
    redis_client: Optional[Redis] = None
    ready: bool = False
```

```
startup_error: Optional[str] = None

app_state = AppState() # singleton – di-import ke seluruh app
```

6. /ready — Runtime Check vs Object Existence

Satu insight penting dari sesi ini: `app_state.redis_client` is not None tidak cukup sebagai health check untuk Redis. Object bisa ada (tidak None) tapi koneksi sudah putus.

Analoginya: punya nomor telepon seseorang tidak berarti mereka masih bisa dihubungi.

```
# SALAH – hanya cek object existence

checks = {
    "database_connected": app_state.redis_client is not None
}

# BENAR – lakukan aktif ping ke server

redis_alive = False

if app_state.redis_client:
    try:
        app_state.redis_client.ping() # kirim PING, tunggu PONG
        redis_alive = True
    except redis.RedisError:
        pass # server tidak merespons – tandai False
```

7. Disk Space Check

Salah satu check yang sering dilupakan di `/ready` adalah disk space. Kalau disk penuh, app tidak bisa menulis log, tidak bisa menyimpan file sementara, dan behavior-nya jadi tidak terduga. Lebih baik deteksi lebih awal via `/ready` daripada crash mendadak.

```
import shutil

def _check_disk_space(min_free_mb: int = 100, path: str = '/') -> bool:
    try:
        total, used, free = shutil.disk_usage(path)
        free_mb = free / (1024 * 1024)
        if free_mb <= min_free_mb:
            logger.warning('readiness_disk_check_failed',
                           extra={'free_mb': round(free_mb, 2), 'path': path})
            return False
        return True
    except Exception as e:
        logger.error('disk_check_error', extra={'error': str(e)}, exc_info=True)
        return False
```

Kenapa `logger.error('disk_check_error', extra={...})` bukan f-string? Karena structured logging mengharuskan event name berupa string statis — agar bisa di-query di log aggregator (Loki, Datadog). F-string menghasilkan string yang unik setiap kali, sehingga tidak bisa di-group.

8. Prinsip Kunci yang Harus Diingat

- **/health** tidak pernah cek dependency. Konsekuensi fail-nya adalah restart — restart tidak membantu kalau yang rusak adalah Redis atau DB eksternal.
- **/ready** cek semua dependency kritis secara aktif (ping, not None check). Fail-nya hanya menghentikan traffic baru — container tetap hidup.
- **app_state.ready = True** harus selalu jadi baris TERAKHIR startup. Jangan biarkan traffic masuk sebelum semua resource siap.
- **app_state.ready = False** harus selalu jadi baris PERTAMA shutdown. Komunikasikan ke load balancer secepat mungkin.
- **try/finally di lifespan** menjamin shutdown code selalu dieksekusi, bahkan kalau ada exception saat app berjalan.
- Semua log event name harus string statis, bukan f-string. Detail masuk ke `extra={}` agar bisa di-query di log aggregator.